Plano completo de implementação — Agentes autônomos

Base: `ESPECIFICACAO_AGENTES_MONITORAMENTO.md`
Execução: uma tarefa por commit; testes antes de cada commit; sem apagar dados do banco compartilhado.

Estratégia

Implementar do núcleo para as bordas: primeiro dados confiáveis e episódios, depois runtime e IA, em seguida ações fake, canais reais e frontend. Nenhuma integração externa será ativada antes de funcionar com executor fake, idempotência e auditoria.

Fase 0 — Preparação e baseline

T0.1 — Ambiente reproduzível

- Consolidar requirements e `.venv`.
- Centralizar configuração sem segredos versionados.
- Criar comandos de teste local e modo dry-run.
- Confirmar portas Tracker 9000, API 5009 e frontend App 2 3003.

Aceite: instalação limpa e subida documentada sem depender do diretório atual.

T0.2 — Corrigir integração básica

- Corrigir imports absolutos de `monitor_api`.
- Remover score inventado/fallback silencioso.
- Garantir que `run.py` inicie o motor correto.
- Adicionar health checks de API, banco, simuladores e monitor runtime.

Aceite: health check identifica claramente cada dependência.

Fase 1 — Dados comportamentais confiáveis

T1.1 — Detecção baseada em tempo

- Usar `Δtempo`, `Δvelocidade` e `Δrumo`.
- Tratar amostras atrasadas, duplicadas e fora de ordem.
- Separar evento bruto de episódio consolidado.

T1.2 — Score com recuperação

- Implementar janela deslizante da última hora.
- Penalizar eventos por gravidade e reincidência.
- Recuperar score quando o comportamento normalizar.
- Testar perfis good, moderate e poor.

T1.3 — Memória e persistência

- Limitar memória por veículo.
- Adicionar locks e fila de persistência.

- Criar índices para veículo, tempo, tipo e estado.
- Nunca bloquear o recebimento de telemetria por uma escrita lenta.

T1.4 — Dirijabem no mesmo modelo

- Mapear `APP-<CODUSU>` para motorista/veículo.
- Converter localização Dirijabem em evento normalizado.
- Manter banco Dirijabem somente leitura fora dos simuladores.

Saída da fase: eventos Tracker e Dirijabem chegam ao mesmo pipeline normalizado.

Fase 2 — Modelo de agentes

T2.1 — Schema e migrations

Criar migrations aditivas para:

- agentes;
- episódios/incidentes;
- memória;
- ciclos de decisão;
- ações;
- mensagens;
- escalonamentos;
- auditoria.

Aceite: migration idempotente e testada em banco local.

T2.2 — Definições iniciais

Cadastrar os agentes de segurança, operação, rota/jornada, manutenção e atendimento. Cada definição terá missão, eventos, ferramentas, autonomia e políticas.

T2.3 — Runtime

Implementar `agent_runtime` para:

1. consumir eventos;
2. agrupar episódios;
3. buscar contexto;
4. aplicar cooldown;
5. solicitar decisão;
6. validar decisão;
7. enfileirar ação;
8. agendar follow-up.

T2.4 — Memória operacional

- Resumir histórico relevante, sem enviar dados ilimitados ao LLM.
- Guardar fatos, decisões e respostas.
- Expirar contexto temporário.
- Garantir isolamento entre veículos e agentes.

Fase 3 — IA externa e fallback

T3.1 — Cliente de IA

Criar `server/llm_client.py` com:

- provider configurável;
- timeout;
- retry limitado;
- logs sem conteúdo sensível;
- token/custo observável;
- desligamento por configuração.

T3.2 — Prompt operacional

O prompt deve conter missão do agente, ferramentas permitidas, fatos do episódio, memória, políticas e contrato JSON. A IA deverá justificar a decisão com IDs de evidência.

T3.3 — Validação e segurança

- Validar JSON com schema.
- Recusar ferramenta não autorizada.
- Recusar ação sem destinatário ou evidência.
- Sanitizar mensagens.
- Não permitir que texto de telemetria altere as instruções do agente.

T3.4 — Fallback

Se a IA falhar, exceder timeout ou devolver formato inválido, usar decisão rule-based equivalente, registrar o motivo e manter o acompanhamento ativo.

Aceite: testes passam com API fake, falha de rede e resposta inválida.

Fase 4 — Fila e executor de ações

T4.1 — Action queue

Implementar fila durável com:

- prioridade;
- prazo;
- tentativas;
- backoff;
- idempotency key;
- cancelamento;
- dead-letter para falhas permanentes.

T4.2 — Conectores fake

Criar conectores de mensagem, e-mail e telefonia fake que registram payload, destinatário e resultado. Eles serão usados em todos os testes end-to-end.

T4.3 — Canais reais

Integrar o provedor escolhido para WhatsApp/SMS e telefonia. As credenciais ficam somente em ambiente seguro. Ativar primeiro em sandbox e allowlist de números.

T4.4 — Respostas e follow-up

- Receber status de entrega.
- Receber respostas de usuário.
- Associar resposta ao incidente.
- Agendar reavaliação.
- Escalar quando não houver resposta.

Fase 5 — Políticas de autonomia

T5.1 — Níveis de autonomia

Implementar níveis 0–4 e validar cada ferramenta contra o nível do agente.

T5.2 — Limites

Implementar cooldown, limite de mensagens, horário de silêncio, contatos secundários e exigência de aprovação para ações críticas.

T5.3 — Auditoria

Registrar prompt/versionamento, modelo, evidências, decisão, ações, resultado e usuário que alterou a política.

T5.4 — Pausa e intervenção humana

Adicionar pausa global, pausa por agente, cancelamento de ação e takeover humano. O agente deve deixar o caso intacto para retomada.

Fase 6 — Frontend operacional

- Remover mocks do App 2.
- Exibir agentes, missão, autonomia e estado.
- Exibir episódio e linha do tempo de decisões/ações.
- Configurar contatos e canais.
- Mostrar aprovação pendente.
- Mostrar mensagens enviadas, respostas e escalonamentos.
- Permitir pausar, retomar, cancelar e assumir manualmente.
- Exibir evidências usadas pela IA.

Fase 7 — Cenários completos de teste

Testes unitários

- detectores e score;
- agrupamento de episódios;
- cooldown;
- schema de decisão;
- autorização de ferramentas;
- cálculo de escalonamento;
- idempotência da fila.

Testes de integração

- Tracker → detector → agente;
- Dirijabem → normalização → agente;
- agente → IA fake → fila;
- fila → conector fake → follow-up;
- falha de IA → fallback;
- falha de canal → retry/escalamento.

Testes end-to-end

1. Excesso de velocidade pontual: registra e não incomoda.
2. Reincidência: orienta motorista e avisa gestor.
3. Persistência sem resposta: liga e escala.
4. Possível colisão: ação crítica imediata.
5. Normalização: encerra com evidência.
6. Falso positivo: agente explica e não abre nova ocorrência.
7. Dirijabem: viagem sintética gera análise e comunicação.

Ordem recomendada de execução

1. T0.1–T0.2: ambiente e inicialização.
2. T1.1–T1.4: eventos e score confiáveis.
3. T2.1–T2.4: entidades e runtime de agentes.
4. T3.1–T3.4: IA externa com fallback.
5. T4.1–T4.4: ações fake, reais e follow-up.
6. T5.1–T5.4: autonomia e guardrails.
7. Fase 6: painel operacional.
8. Fase 7: validação completa e calibração de 30 minutos.

Definition of Done

- Um agente acompanha um episódio sem intervenção manual no código.
- A decisão da IA é estruturada, validada e auditável.
- Ações são executadas por fila idempotente.
- Mensagem e ligação possuem resultado registrado.
- Ausência de resposta causa follow-up e escalonamento.
- Fallback funciona sem IA externa.
- Dirijabem e Tracker usam o mesmo ciclo operacional.
- Frontend não depende de mock nos fluxos principais.
- Testes unitários, integração e end-to-end passam.
- Nenhuma credencial ou alteração destrutiva foi incluída.